FICHE MÉMO

Mémo du module 0

Les notions du module, dans l'ordre du deck : la mécanique d'un LLM, le contexte, les embeddings et le RAG, le prompt structuré, puis les outils Claude et leurs règles d'usage.

> **Takeaway** : un contexte clair + des instructions précises + des données fiables = une IA utile et de confiance.

Objectifs du bootcamp

1. **Comprendre la mécanique d'un LLM** (tokens, probabilités, attention, context window) pour prompter plus finement et diagnostiquer les erreurs.
2. **Utiliser Claude au quotidien** dans ses versions chat, Projects et Cowork, en choisissant la bonne selon le besoin.
3. **Identifier les limites et poser les rails** : hallucinations, coût des tokens, données sensibles, Shadow AI, Zero-Data Retention, RAG, validation DSI.
4. **Manipuler, pas seulement écouter**, à travers plusieurs labs : Project, tâche récurrente, Cowork.

1. Un domaine en plein essor

Les domaines de l'IA

Domaine	En une phrase
Intelligence Artificielle	Automatisation informatisée de tâches habituellement réalisées par des humains
Machine Learning	Un algorithme « apprend » à partir de données (Markov, régressions, classification de contenu)
Deep Learning	Apprentissage par réseaux de neurones, volumineux et coûteux
GenAI, LLM	Inclus dans le Deep Learning : les LLM sont une forme de GenAI
Natural Language Processing	Reconnaissance du langage naturel, recoupe ces domaines

Mécanique des LLM : tokens, probabilités, température

- Un LLM découpe le texte en **tokens** (~0,75 mot en français, unité de mémoire et de facturation) et prédit, à chaque étape, le token suivant le plus **probable**.
- La **température** règle le degré d'aléa de ce choix : basse pour le code, haute pour la créativité. Le **top-p** restreint le choix aux p % de masse de probabilité.
- Le LLM ne comprend pas au sens humain : il prédit le prochain token.

> **Conséquence pratique** : deux prompts identiques peuvent donner deux réponses différentes. C'est une fonctionnalité, pas un bug.

La mémoire des LLM

La « mémoire » d'un assistant est une **répétition dans le prompt** : les préférences sont stockées puis réinjectées au début de chaque requête par un prompt système invisible. Les poids et l'architecture du LLM restent strictement identiques.

Mémoire de travail : contexte, compression et lost-in-the-middle

Notion	Ce qu'il faut savoir
Context Window	Quantité de tokens vue en une requête : prompt + historique + pièces jointes + réponse. Claude Sonnet 4.6 va jusqu'à 1M tokens, mais précision et coût se dégradent bien avant ce plafond.
Context Compression	Quand la fenêtre sature (longue session Claude Code, Cowork…), les anciens échanges sont résumés automatiquement. On garde l'essentiel, on perd les détails.
Lost in the middle	L'attention n'est pas uniforme : le modèle retient mieux le début et la fin (courbe en U).

Bonnes pratiques :

- Instructions critiques **au début ET rappelées à la fin** du prompt.
- Documents : les plus importants en premier.
- Découper les longues missions en étapes courtes.
- Sur un travail long : « résume ce qu'on a fait, et continue ».

Agentique

L'outil (le programme autour du LLM) reçoit la requête, signale au LLM les agents disponibles (recherche…), exécute les appels demandés et renvoie les résultats au LLM : **plusieurs requêtes successives** pour une seule question. Trois familles d'accès : RAG et données locales, outils via **MCP**, réalisation d'actions.

Embeddings et RAG

- **Embeddings** : un texte est projeté en **vecteur numérique** dans un espace à plusieurs centaines de dimensions. Deux textes proches par le sens se retrouvent proches dans l'espace, même sans mots communs : « bug en prod » ≈ « incident critique » ≈ « panne système ».
- **RAG** (Retrieval-Augmented Generation) : répondre à partir de **vos documents** sans les avoir appris.

Étape	Ce qui se passe
1. Indexation	Les documents sont découpés en chunks, transformés en embeddings, rangés dans une base vectorielle
2. Retrieval	Pour une question, on récupère les chunks les plus proches sémantiquement

Étape	Ce qui se passe
3. Augmentation	Les chunks retrouvés sont injectés dans le prompt comme contexte

> C'est la parade n°1 contre les **hallucinations**. Claude : les Projects font du RAG léger automatiquement ; au-delà (10k+ docs), un connecteur MCP vers une vector DB.

L'écosystème et les règles d'utilisation

- **Assistants bureautique** (intégrés aux applications), **assistants de dév** (Dev / Sec / Ops), **modèles et Cloud** (on-prem ou via des providers).
- Toujours utiliser une IA générative **contractualisée** par votre entreprise : pas de réutilisation des données pour l'entraînement (**Zero-Data Retention**) et confidentialité garantie. Pour tout autre outil : demander à la **DSI**.
- **Shadow AI** : un outil non contractualisé expose vos données à leur réutilisation.
- **Nos postures comptent** : individu → équipe → entreprise ; l'humain reste au centre ; une utilisation à bon escient, pas masquée, critique.

2. L'art du prompt avec Claude

Les réglages de Claude

- **Choisir le modèle** (menu en haut du chat) : Opus pour raisonner, Sonnet par défaut, Haiku pour aller vite.
- **Activer des capacités** : recherche web, exécution de code, lecture de fichiers joints (PDF, xlsx, images), connecteurs MCP (Drive, Slack, Asana…), génération d'Artifacts.
- **Ranger les conversations par Projects** : un historique + des fichiers + une instruction persistante.
- Garder en tête le **coût** et la **sobriété**.

Le prompt structuré

Élément	Exemple
Persona	« Tu es un expert commercial en IT… »
Tâche	« Tu dois produire une trame de réponse pour cet appel d'offre… »
Format	« …au format Markdown, ton professionnel, sans réexpliquer les mots de l'appel d'offre… »
Contexte	le client, l'évènement, le mail contenant…

> **Style** : Claude abuse du gras et des listes. Utiliser des **balises XML** (`<contexte>` , `<tache>` , `<exemple>`) pour isoler proprement les blocs du prompt.

Styles et Projects

- **Styles** : ajuster le ton et le format (concis, formel, explicatif…), présets ou personnalisés.
- **Projects** : un espace de travail persistant (documents de référence, instructions, historique), partageable au sein d'une **Team Anthropic**.

3. Outils, usages et mise en pratique

Outil	Ce qu'il fait
Artefacts	Un livrable (document, tableau, page HTML, mini-app) ouvert dans un panneau à droite : édition itérative, export Word / Excel / PDF / code, persistance
Claude Opus 4.7	Le raisonneur profond : tâches complexes, analyses longues. Plus lent, plus cher
Claude Sonnet 4.6	Le couteau suisse, par défaut : 90 % des usages
Claude Haiku 4.5	Le rapide : tâches simples et volumineuses. Quasi-instantané, très bon marché
Connecteurs MCP	Protocole standard pour lire / écrire dans un outil externe (Drive, Slack, Asana, Jira, GitHub…)
Claude in Chrome (beta)	Pilote vos onglets : formulaire, extraction, navigation
Claude in Excel (beta)	Agent tableur : nettoyer, formules, rapport
Cowork	Agent de bureau avec accès contrôlé à un dossier local et au navigateur, pour les profils non techniques
Claude Code	Agent dans le terminal qui pilote un dépôt comme un développeur, pour les profils tech

- **Pensée adaptative** : sur Opus et Sonnet, un toggle laisse Claude « réfléchir avant de répondre ».
- **Skill** : un dossier qui emballe un savoir-faire reproductible, un `SKILL.md` (mode d'emploi) + des fichiers de support, chargé **automatiquement** quand il correspond à la demande. Structure minimale : un nom + une description (quand l'activer), un `SKILL.md` (étapes, règles, pièges), en option des templates / checklists.
- **Project vs Skill** : un Project porte un **contexte** (« tout sur le client X »), une Skill porte une **compétence** (« comment rédiger un CR chez nous »). Les deux se combinent.
- **Cowork vs chat** : Cowork a les mains dans vos fichiers et vos applis ; le chat se limite aux pièces jointes uploadées.
- **Claude Code** : naviguer dans un dépôt, écrire du code sur plusieurs fichiers, lancer les tests, interagir avec Git, ouvrir une PR, appeler des MCP, charger des Skills, travailler en **mode plan** ou en exécution directe.

> Chaque intégration doit être validée par la **DSI** : pas de données client sans feu vert.

4. Synthèse : les 4 rails d'une IA utile

Rail	Ce qu'on fait
Contexte	Projects pour persister, pièces jointes pour ponctuel, RAG pour la masse
Instructions	Persona, Tâche, Format, Contexte. Balises XML pour structurer. Critiques en haut et en bas du prompt
Modèle et réglages	Haiku pour la vitesse, Sonnet par défaut, Opus pour raisonner. Température basse pour le code, haute pour la créa
Vérification	Citations (RAG), relire le diff (Claude Code), ne jamais oublier l'hallucination. Règle DSI sur les données sensibles

> L'écosystème change constamment : le plus important est de **continuer à pratiquer**.

SUPPORT D'EXERCICE

Lab 1 : prompting professionnel

Deux temps : créer votre premier Project Claude sur vos propres documents, puis passer de la question jetée au chat à un prompt structuré, répétable et transférable, sur un use case de support e-commerce.

Partie A : Mon premier Project Claude

Objectif : produire un livrable métier concret (synthèse, mail, note de cadrage) à partir de vos propres documents, rangé dans un Project réutilisable.

1. **Créer un Project dédié** à un sujet précis, avec un nom clair.
2. **Ajouter 3 à 5 documents** personnels (notes, mails, slides, charte…), *pas de données strictement confidentielles*.
3. **Renseigner les instructions** du Project : rôle de Claude, ton, tabous, citations attendues.
4. **Poser trois questions** et demander un livrable concret.
5. **Itérer** : demander les passages cités, corriger, ajuster le rôle.

> Pièges fréquents : documents trop volumineux, données confidentielles, hallucinations (toujours redemander les citations).

Partie B : Prompting professionnel (~60 min)

Objectif : passer de la question jetée au chat à un prompt **structuré, répétable et transférable**, qu'on pourrait donner à un collègue.

1. **Rôles (Persona)** : « Tu es un… » avec expertise et posture.
2. **Contraintes** : format de sortie, longueur, style, interdits.
3. **Few-shot prompting** : coller 2-3 exemples de la bonne réponse. Ce qui fait passer de « correct » à « on dirait nous ».
4. **Chain-of-Thought** : raisonner étape par étape avant de conclure. Utile pour l'analyse et le code, inutile sur les tâches simples.
5. **Gérer la context compression** : redonner les règles clés, fournir un résumé, repartir d'un prompt propre plutôt qu'empiler.
6. **Tester, critiquer, itérer** : un bon prompt est un artefact qu'on maintient.

> **Règle d'or** : le prompt final est un livrable réutilisable, rangez-le dans un Project ou une Skill.

Use case : support e-commerce

Fournir un prompt générique et fiable pour le support d'une entreprise e-commerce répondant à un client insatisfait.

Message client :

> « Franchement très déçu. Livraison en retard de 5 jours, produit abîmé à l'arrivée, et aucune réponse du support. Je veux un remboursement immédiat. »

Contexte entreprise :

- Marque premium mais accessible.
- Ton : professionnel, empathique, jamais familier.
- Politique : remboursement ou renvoi sous 48 h.
- Objectif : fidéliser malgré l'incident.

La trame du prompt

Les quatre éléments vus en séance, chacun dans sa balise XML (Claude abuse du gras et des listes : les balises isolent proprement les blocs).

Élément	À renseigner
Persona	le rôle et la posture de Claude
Tâche	ce qu'il doit produire
Format	forme, longueur, ton, interdits
Contexte	le message client, la politique, l'objectif

```text
<persona>…</persona>
<tache>…</tache>
<format>…</format>
<contexte>…</contexte>
<exemple>…</exemple>
```

Exemple de la démo : le discours des 60 ans

Le prompt structuré montré en démo, à reprendre comme modèle.

text

```
<persona>
Tu es un proche qui prend la parole en public, sur un ton chaleureux,
un brin humoristique mais jamais moqueur. Tu doses l'émotion sans tomber
dans le pathos.
</persona>

<tache>
Rédige-moi un discours de 3 minutes pour les 60 ans de mon père,
à lire devant la famille et une quinzaine d'amis proches lors du dîner.
</tache>

<format>
Une intro qui pose la soirée, deux anecdotes marquantes, un mot sincère,
une chute qui lance le toast.
Phrases courtes, pensées pour être lues à voix haute.
</format>

<contexte>
  <anecdotes>
    1. Apprentissage du vélo : été 1998, la côte derrière la maison ;
       il a couru à côté sans jamais lâcher le vélo avant que je le lui dise.
    2. Pompiers volontaires pendant 12 ans : deux interventions marquantes,
       dont un feu de grange le soir de Noël.
    3. Voyage aux Lofoten : tente sous -5 °C, il a fait rire toute la troupe
       pendant une semaine.
    4. [anecdote 4]
    5. [anecdote 5]
  </anecdotes>
  <caractere>
    Bricoleur, pudique sur ses émotions, fidèle en amitié,
    taquin avec ses petits-enfants.
  </caractere>
  <passions>
    La voile, le jardinage, les polars scandinaves, le rugby.
  </passions>
  <tonalite_soiree>
    Dîner décontracté à la maison, une quinzaine de personnes,
    ambiance familiale. Pas de discours solennel : on veut de la chaleur
    et des rires.
  </tonalite_soiree>
</contexte>
```

Pour les usages récurrents, un **Project Claude** capitalise sur son style et son contexte sans les réécrire.

SUPPORT D'EXERCICE

Lab 2 : automatiser une tâche récurrente

Lab optionnel (~25 min) : transformer une tâche faite « à la main » chaque semaine en un Project ou une Skill réutilisable en 30 secondes.

Étapes

1. **Choisir une tâche récurrente** : CR de réunion, relance client, brief offre, réponse candidat…
2. **Créer un Project / une Skill** dédié, avec un nom clair.
3. **Rédiger les instructions** : destinataire, ton, format de sortie, quoi éviter.
4. **Ajouter 2-3 exemples** en pièces jointes, ce qui fait passer de « correct » à « on dirait vous ».
5. **Tester sur un cas réel** et affiner selon ce qui manque ou dérape.

> **Règle d'or** : pas de données client confidentielles sans validation DSI, données anonymisées ou cas internes.

Project ou Skill ?

	Project	Skill
Porte	un **contexte**	une **compétence** reproductible
Exemple	« tout sur le client X »	« comment rédiger un CR chez nous »
Contenu	documents de référence, instructions, historique	un `SKILL.md` (mode d'emploi) + des fichiers de support
Chargement	quand on travaille dans le Project	**automatiquement**, quand la Skill correspond à la demande
Partage	au sein d'une Team Anthropic	au sein d'une Team Anthropic, comme un Project

Les deux se combinent.

Structure minimale d'une Skill

- un **nom** + une **description** (quand l'activer) ;
- un `SKILL.md` (étapes, règles, pièges) ;
- en option des templates / checklists.

Pour la créer : demandez à Claude « je veux créer une nouvelle skill », ou passez par le menu **Customize**.

```text
Je veux créer une nouvelle skill.
```

SUPPORT D'EXERCICE

Lab 3 : prendre en main Cowork

Lab optionnel (~25 min) : déléguer à Cowork la préparation d'un livrable à partir de fichiers dispersés sur votre poste.

Rappel : Claude Cowork

Cowork (beta, research preview) est un agent Claude qui tourne sur votre poste avec un accès contrôlé à un dossier local et au navigateur. Pensé pour les profils non techniques : tout en langage naturel.

- Lire et organiser les fichiers d'un dossier (nommer, ranger, archiver).
- Extraire du contenu d'un PDF, d'une image, d'un fichier Office.
- Créer des documents Word, Excel, PowerPoint à partir d'un brief.
- Exécuter une **Skill** sur vos données.
- Se connecter à vos outils (Slack, Drive, GitHub…) via MCP.

Différence avec le chat : Cowork a les mains dans vos fichiers et vos applis ; le chat se limite aux pièces jointes uploadées.

Étapes

1. **Préparer un dossier dédié** (ex. `~/bootcamp-lab`) avec 4-5 fichiers réels non confidentiels.
2. **Ouvrir Cowork** et sélectionner ce dossier comme espace de travail.
3. **Donner une mission** :

```text
Lis tous les fichiers, résume les points clés dans un Word,
et range les fichiers par date.
```

4. **Observer** Cowork lire, demander confirmation, produire, et corriger au fil de l'eau.
5. **Itérer** :

```text
Même synthèse en slides.
```

```text
Extrais les actions dans un Excel.
```

> **Règle d'or** : isoler un dossier dédié, relire chaque modification, pas de dossier client confidentiel sans validation DSI.

> **Astuce** : faites créer par Claude les documents d'une fausse entreprise (fiche produit, facture, mails, slides).

SUPPORT D'EXERCICE

Lab 4 : prendre en main Claude Code

Lab optionnel (~30 min) : confier à Claude Code une tâche concrète sur un dépôt réel.

Rappel : Claude Code

Claude Code est un agent Claude qui tourne dans votre terminal et pilote votre dépôt comme un développeur. Pour les profils tech, installable en une commande npm.

- Naviguer dans un dépôt (même gros) avec le bon niveau de contexte.
- Écrire du code, modifier plusieurs fichiers, lancer les tests, corriger les erreurs.
- Exécuter des commandes shell, interagir avec Git, ouvrir une PR.
- Appeler des **MCP**, charger des **Skills** dédiées (SRE, DB, review, tests…).
- Fonctionner en **mode plan** ou en exécution directe.

Où ça brille : refactor multi-fichiers, migration de framework, génération de tests sur du legacy, exploration d'un dépôt inconnu, investigation d'un bug via les logs.

Étapes

1. **Choisir un petit dépôt** (perso ou side-project anonymisé). Installer Claude Code et le lancer à la racine du dépôt :

```bash
npm install -g @anthropic-ai/claude-code
claude
```

2. **Laisser explorer** : `/init` génère un `CLAUDE.md` (architecture, conventions, commandes).

```text
/init
```

3. **Choisir une mission de 15-30 min** : ajouter un test, renommer une fonction partout, documenter un module, corriger une typo dans 10 fichiers.
4. **Demander un plan** (mode plan) puis laisser exécuter. Observer : Claude Code lit, modifie, teste, explique.
5. **Relire le diff**, accepter / refuser, ouvrir une PR.

> **Règle d'or** : dépôt non critique, pas de secrets dans l'historique, toujours relire le diff avant merge. Pas de dépôt client sans validation DSI.